



Hochiminh City University of Technology
Computer Science and Engineering
[CO1027] - Fundamentals of C++ Programming

Vector, Iterator

Lecturer: Duc Dung Nguyen
Credits: 3

Outcomes

- ❖ Able to use vector and iterator

Outline

- ❖ Vector
- ❖ Iterator

Drawbacks to Arrays

- ❖ The size of the array must be a fixed constant, not a variable.
- ❖ Passing an array as a parameter to a function along with its size.
- ❖ No way to conveniently insert elements at the beginning or in the middle of an array.
- ❖ Similarly, no way to conveniently remove an element from an array
- ❖ Cannot return an array from a function.

Vectors

- ❖ C++ Standard Library class template **vector**, which represents a more robust type of array featuring many additional capabilities.
- ❖ Vectors are sequence containers representing arrays that can change in size.

Advantages of Vectors

- ❖ The size of a vector does not have to be a fixed constant, and it can also grow or shrink during execution
- ❖ A vector always knows its own size, so passing one to a function does not require to separately pass this information.
- ❖ Elements can be accessed by position in a vector just as they are in an array, but you can also insert and remove elements anywhere in a vector.
- ❖ Vectors can be returned from a function easily.

Iterator

- ❖ Vectors are more versatile than arrays, and can be resized during runtime. To keep track of where we are in the vector, we need an **iterator**.
- ❖ Creating an iterator for the vector
 - ❖ `vector<int>::iterator it;`
- ❖ Using the iterator to cycle through the vector. Begin is the first element in the vector, end is the last.
 - ❖ `vector<int>::iterator it;`
`for (it = a.begin(); it!=a.end(); it++)`
`cout << *it << " ";`

Assign Elements

- ❖ vector::assign: assigns new contents to the vector, replacing its current contents, and modifying its size accordingly.

```
vector<int> a;  
cout << a.size()<<endl;  
a.assign(3, 5);  
cout << a.size()<<endl;
```

Push_back Element

- ❖ `vector::push_back`: adds a new element at the end of the vector, after its current last element. This effectively increases the container size by one, which causes an automatic reallocation of the allocated storage space if -and only if- the new vector size surpasses the current vector capacity.

```
vector<int> a;  
a.assign(3, 5);  
a.push_back(15);
```

Pop_back Element

- ❖ `vector::pop_back`: removes the last element in the vector, effectively reducing the container size by one.
- ❖ This destroys the removed element.
`vector<int> a;`
`a.assign(3, 5);`
`a.push_back(15);`

Insert Element

- ❖ `vector::insert` : The vector is extended by inserting new elements before the element at the specified *position*, effectively increasing the container size by the number of elements inserted. This causes an automatic reallocation of the allocated storage space if -and only if- the new vector size surpasses the current vector capacity.

```
vector<int>::iterator it = a.begin();  
a.insert(it + 2, 100);  
print(a);
```